

Taking Input from User in R

In R, we use `readline()` function to take input from user.

Example 1: Simple Input

```
name <- readline(prompt = "Enter your name: ")
print(name)
```

Output:

```
Enter your name: Neeraj
[1] "Neeraj"
```

Example 2: Numeric Input

Important: `readline()` gives **character (string)**
So we convert it into numeric

```
num <- as.numeric(readline(prompt = "Enter a number: "))
print(num * 2)
```

Example 3: Two Numbers Addition

```
a <- as.numeric(readline(prompt = "Enter first number: "))
b <- as.numeric(readline(prompt = "Enter second number: "))

sum <- a + b
print(sum)
```

Important Points

- `readline()` always returns **character**
- Use **type casting** (`as.numeric`) if needed
- Works in **R console / terminal**

2. Functions in R

A **function** is a block of code that performs a specific task and can be reused.

Why Functions?

- Code reuse
- Easy to manage
- Reduces repetition

Syntax of Function

```
function_name <- function(parameters) {
  # code
  return(value)
}
```

Example 1: Simple Function

```
greet <- function() {  
  print("Hello, R!")  
}  
  
greet()
```

Example 2: Function with Parameters

```
add <- function(a, b) {  
  result <- a + b  
  return(result)  
}  
  
print(add(10, 5))
```

Example 3: Function without return()

R automatically returns last value

```
square <- function(x) {  
  x * x  
}  
  
print(square(4))    # 16
```

Example 4: Default Arguments

```
greet <- function(name = "User") {  
  print(paste("Hello", name))  
}  
  
greet()              # Hello User  
greet("Neeraj")      # Hello Neeraj
```

Example 5: Function with User Input

```
add_numbers <- function() {  
  a <- as.numeric(readline("Enter first number: "))  
  b <- as.numeric(readline("Enter second number: "))  
  
  print(a + b)  
}  
  
add_numbers()
```

Example 6: Multiple Values (List)

```
calc <- function(a, b) {  
  sum <- a + b  
  diff <- a - b  
  
  return(list(sum = sum, difference = diff))  
}  
  
result <- calc(10, 5)  
print(result$sum)  
print(result$difference)
```

Built-in Functions in R

R already has many functions:

```
sum(1, 2, 3)
mean(c(1, 2, 3))
sqrt(16)
length(c(1, 2, 3))
```

String Handling in R

A **string** is a sequence of characters (text)

In R, strings are written inside:

- " " (double quotes)
- ' ' (single quotes)

Example:

```
name <- "Neeraj"
city <- 'Lucknow'
```

1. **paste()** Function

Used to **join (concatenate) multiple strings**

Syntax

```
paste(text1, text2, sep=" ")
```

Examples

```
first <- "Neeraj"
last <- "Katheriya"
```

```
paste(first, last)
```

Output:

```
"Neeraj Katheriya"
```

Without Space

```
paste(first, last, sep = "")
```

Output:

```
"NeerajKatheriya"
```

Multiple Values

```
paste("R", "Programming", "Language")
```

Output:

```
"R Programming Language"
```

2. **nchar()** Function

Used to **count number of characters** in a string

Example

```
name <- "Neeraj"  
nchar(name)
```

Output:

```
6
```

3. toupper() and tolower()

Used to **change case of text**

toupper() → Convert to UPPERCASE

```
x <- "hello"  
toupper(x)
```

Output:

```
"HELLO"
```

tolower() → Convert to lowercase

```
x <- "HELLO"  
tolower(x)
```

Output:

```
"hello"
```

4. substr() Function

Used to **extract part of a string**

Syntax

```
substr(text, start, stop)
```

Example

```
name <- "Neeraj"  
  
substr(name, 1, 3)
```

Output:

```
"Nee"
```

Another Example

```
text <- "R Programming"  
  
substr(text, 3, 12)
```

Output:

```
"Programming"
```

Summary

Function	Use
<code>paste()</code>	Join strings
<code>nchar()</code>	Count characters
<code>toupper()</code>	Convert to uppercase
<code>tolower()</code>	Convert to lowercase
<code>substr()</code>	Extract part of string

Data Visualization in R

1. `plot()` — Basic Scatter Plot

Used to study **relationship between two continuous variables**

Research Use:

- Gene expression vs time
- Enzyme activity vs substrate concentration
- Dose-response experiments

Example: Gene Expression Time-Course

```
# Time (hours)
time <- c(0, 2, 4, 6, 8, 10)

# Gene expression (relative units)
expression <- c(2, 5, 9, 15, 22, 30)

plot(time, expression,
      main = "Gene Expression Over Time",
      xlab = "Time (hours)",
      ylab = "Expression Level",
      pch = 19,
      col = "blue")

# Add trend line (important for research)
model <- lm(expression ~ time)
abline(model, col = "red", lwd = 2)
```

2. `barplot()` — Basic Comparison

Used for **comparing groups**

Research Use:

- Control vs treated samples
- Drug comparison

- Protein expression levels

Example: Drug Effect on Cell Viability

```
viability <- c(95, 70, 50)
groups <- c("Control", "Drug A", "Drug B")

barplot(viability,
        names.arg = groups,
        col = c("green", "orange", "red"),
        main = "Cell Viability After Drug Treatment",
        ylab = "Viability (%)")

# Add values on top (important for presentation)
text(x = 1:3, y = viability, label = viability, pos = 3)
```

3. pie() — Distribution

Shows **proportion of categories**

Research Use:

- Cell type composition
- Microbial population
- Tissue distribution

Example: Cell Type Distribution

```
cells <- c(40, 35, 25)
labels <- c("T-cells", "B-cells", "NK-cells")

pie(cells,
    labels = labels,
    main = "Immune Cell Distribution",
    col = c("skyblue", "pink", "lightgreen"))
```

Advanced (Very Important for Research Papers)

Use **ggplot2** (publication-quality graphs)

```
install.packages("ggplot2")
library(ggplot2)
```

4. geom_point() — Scatter Plot (Advanced)

Best for **correlation studies**

Example: Enzyme Kinetics (Concentration vs Activity)

```
df <- data.frame(
  concentration = c(1, 2, 3, 4, 5),
  activity = c(2, 6, 11, 15, 18)
)

ggplot(df, aes(x = concentration, y = activity)) +
  geom_point(size = 3, color = "blue") +
```

```
ggtitle("Enzyme Activity vs Substrate Concentration") +
xlab("Concentration (mM)") +
ylab("Activity (Units)") +
theme_minimal()
```

5. geom_line() — Growth Curve

Shows **change over time**

Example: Bacterial Growth Curve

```
df <- data.frame(
  time = c(0, 2, 4, 6, 8, 10),
  growth = c(0.1, 0.5, 1.2, 2.5, 3.8, 4.0)
)

ggplot(df, aes(x = time, y = growth)) +
  geom_line(color = "darkgreen", size = 1.2) +
  ggtitle("Bacterial Growth Curve") +
  xlab("Time (hours)") +
  ylab("Optical Density (OD600)") +
  theme_minimal()
```

6. geom_bar() — Publication-Level Bar Chart

Used for **gene expression / experimental comparison**

Example: Gene Expression (Control vs Treated)

```
df <- data.frame(
  condition = c("Control", "Treated"),
  expression = c(5, 12)
)

ggplot(df, aes(x = condition, y = expression, fill = condition)) +
  geom_bar(stat = "identity") +
  ggtitle("Gene Expression Analysis") +
  ylab("Relative Expression") +
  theme_minimal()
```

ADVANCED (Research-Level)

Add error bars (VERY IMPORTANT in biotech)

```
df <- data.frame(
  condition = c("Control", "Treated"),
  mean = c(5, 12),
  sd = c(1, 2)
)

ggplot(df, aes(x = condition, y = mean, fill = condition)) +
  geom_bar(stat = "identity", width = 0.5) +
  geom_errorbar(aes(ymin = mean - sd, ymax = mean + sd), width = 0.2) +
  ggtitle("Gene Expression with Error Bars") +
  ylab("Expression Level") +
  theme_minimal()
```

Data Manipulation in R (**dplyr** Package)

Data manipulation means:

- Cleaning data
- Filtering useful data
- Transforming variables
- Summarizing results

In research (especially biotech), this is used for:

- Gene expression datasets
- Clinical trial data
- Lab experiment results

Install & Load Package

```
install.packages("dplyr")  
library(dplyr)
```

Sample Dataset (Biotech Example)

We'll use this dataset for all functions:

```
data <- data.frame(  
  sample_id = 1:6,  
  gene = c("BRCA1", "BRCA1", "TP53", "TP53", "EGFR", "EGFR"),  
  condition = c("Control", "Treated", "Control", "Treated", "Control",  
  "Treated"),  
  expression = c(5, 12, 8, 15, 10, 18)  
)  
  
print(data)
```

1. **filter()** — Select Rows Based on Condition

Used to extract **specific data**

Example: Only Treated Samples

```
treated_data <- filter(data, condition == "Treated")  
print(treated_data)
```

Research Use:

- Compare **treated vs control**
- Extract specific experiment group

2. **select()** — Select Columns

Used to choose **important variables**

Example: Only gene and expression


```
selected_data <- select(data, gene, expression)
print(selected_data)
```

Research Use:

- Focus on **important variables only**

3. mutate() — Create New Column

Used to **add new calculated variables**

Example: Log Transformation (very common in biotech)

```
data <- mutate(data, log_expression = log(expression))
print(data)
```

Research Use:

- Normalize gene expression
- Reduce variability

4. arrange() — Sort Data

Used to **sort values**

Example: Sort by Expression (High to Low)

```
sorted_data <- arrange(data, desc(expression))
print(sorted_data)
```

Research Use:

- Identify **highly expressed genes**

5. group_by() — Group Data

Used before summarizing

6. summarise() — Summary Statistics

Used to calculate:

- Mean
- Sum
- Count

Example: Mean Expression per Gene

```
summary_data <- data %>%
  group_by(gene) %>%
  summarise(mean_expression = mean(expression))

print(summary_data)
```

Research Use:

- Average gene expression
- Group comparison

Why **dplyr** is Powerful:

- Clean syntax
- Fast processing
- Easy to read

Real-Life Use in Biotech

- RNA-seq data cleaning
- Differential gene expression
- Clinical data analysis
- Drug response analysis

Final Summary

Function	Use
<code>filter()</code>	Select rows
<code>select()</code>	Select columns
<code>mutate()</code>	Create new column
<code>arrange()</code>	Sort data
<code>group_by()</code>	Group data
<code>summarise()</code>	Summary stats